

KITTI LiDAR Floor/Wall Segmentation 원리 정리

Velodyne point cloud 기반 바닥, 좌/우/정면 벽 후보 분류

LC-EKF VIO project notes

2026-05-04

차 례

1	왜 LiDAR로 나누었나	2
2	입력 데이터와 좌표계	3
3	전체 처리 파이프라인	4
3.1	ROI 필터	4
3.2	평면 모델	4
3.3	RANSAC	5
4	라벨별 판단 기준	6
4.1	Floor	6
4.2	Left/Right Wall	6
4.3	Front Wall	6
5	색상과 Rerun 출력	8
6	코드 위치	9
7	실행 명령	10
8	직접 수집한 데이터로 응용하기	11
8.1	수집해야 하는 데이터	11
8.2	권장 폴더 구조	11
8.3	LiDAR bin 변환 규칙	12
8.4	Timestamp와 동기화	12
8.5	Config 작성	13
8.6	실행	13
8.7	현장 수집 체크리스트	13
9	현재 한계	14

제 1 장

왜 LiDAR로 나누었나

이번 구현의 목적은 KITTI raw 데이터에서 바닥, 좌측 벽, 우측 벽, 정면 벽 후보를 Rerun에서 색으로 확인하는 것이다.

기존 VIO frontend는 stereo image에서 sparse feature를 추적한다. 이 feature map은 pose 추정에는 충분하지만, 면을 안정적으로 분할하기에는 점 밀도가 낮고 표면 전체를 덮지 않는다. 반면 KITTI raw에는 각 camera frame과 동기화된 Velodyne point cloud가 있으므로, 3D geometry를 직접 사용하면 바닥과 벽 후보를 더 단순하고 안정적으로 분리할 수 있다.

현재 구현은 semantic segmentation이 아니다. 딥러닝 모델 없이 3D point cloud의 위치, 평면성, 법선 방향만 이용하는 geometry 기반 plane segmentation이다.

제 2 장

입력 데이터와 좌표계

KITTI Velodyne .bin 파일은 점 하나당 네 개의 float로 저장된다.

```
x y z intensity
```

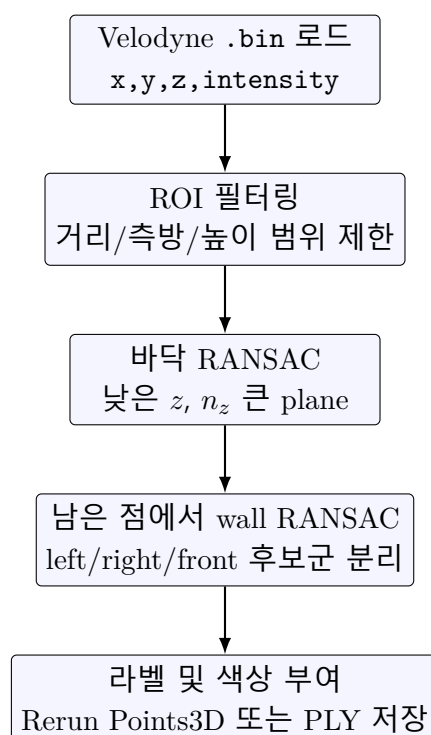
Velodyne 좌표계는 다음 기준으로 사용했다.

축	의미
x	차량 전방
y	차량 좌측
z	위쪽

따라서 바닥은 대체로 낮은 z 에 있고 법선이 z 축에 가깝다. 좌/우 벽은 수직면이므로 법선의 z 성분이 작고, 좌우 방향 벽은 법선이 주로 y 축 방향이다. 정면 벽은 법선이 주로 x 축 방향이다.

제 3 장

전체 처리 파이프라인



3.1 ROI 필터

먼저 너무 멀거나 의미 없는 높이의 점을 제외한다. 기본 범위는 다음과 같다.

항목	기본값	의미
x	$[-5, 50]$ m	차량 뒤쪽 일부와 전방 50 m까지 사용
y	$[-25, 25]$ m	좌우 25 m 범위 사용
z	$[-3, 3]$ m	너무 낮거나 높은 점 제거

3.2 평면 모델

모든 plane fitting은 다음 평면식을 사용한다.

$$\mathbf{n}^T \mathbf{p} + d = 0, \quad \|\mathbf{n}\| = 1$$

세 점 $\mathbf{p}_a, \mathbf{p}_b, \mathbf{p}_c$ 를 뽑으면 법선은 cross product 로 계산한다.

$$\mathbf{n} = \frac{(\mathbf{p}_b - \mathbf{p}_a) \times (\mathbf{p}_c - \mathbf{p}_a)}{\|(\mathbf{p}_b - \mathbf{p}_a) \times (\mathbf{p}_c - \mathbf{p}_a)\|}$$

점 $\mathbf{p}$ 가 평면에 속하는지는 거리로 판단한다.

$$\text{dist}(\mathbf{p}, \Pi) = |\mathbf{n}^T \mathbf{p} + d|$$

거리 threshold 안에 들어오는 점을 inlier 로 센다.

3.3 RANSAC

RANSAC은 다음 반복을 수행한다.

1. 후보 점에서 세 점을 무작위로 선택한다.
2. 세 점이 만드는 평면을 계산한다.
3. 그 평면의 법선 방향이 원하는 class 조건과 맞는지 검사한다.
4. 모든 후보 점에 대해 평면 거리 threshold 이내인 inlier 를 센다.
5. 가장 많은 inlier 를 가진 평면을 채택한다.

구현에서는 deterministic하게 같은 결과가 나오도록 고정 seed 를 사용했다.

제 4 장

라벨별 판단 기준

4.1 Floor

바닥은 낮은 z 범위에서 후보를 모은 뒤, 법선이 위쪽에 가까운 평면을 찾는다.

$$|n_z| \geq 0.85$$

또한 원점 근처의 바닥 높이

$$z_0 = -\frac{d}{n_z}$$

가 floor search 범위 안에 있어야 한다.

4.2 Left/Right Wall

좌/우 벽 후보는 floor로 라벨링되지 않은 점 중 수직 구조일 가능성이 있는 점에서 찾는다.

- left wall 후보: $y \geq 2.0$ m
- right wall 후보: $y \leq -2.0$ m
- vertical plane 조건: $|n_z| \leq 0.25$
- side wall 방향 조건: $|n_y| \geq 0.70$

추가로 너무 작은 cluster가 벽으로 잡히지 않도록, inlier 수와 horizontal/vertical extent 조건을 검사한다.

4.3 Front Wall

정면 벽은 전방 영역의 수직면을 찾는다.

- 후보: $x \geq 5.0$ m, $|y| \leq 12.0$ m

- vertical plane 조건: $|n_z| \leq 0.25$
- front wall 방향 조건: $|n_x| \geq 0.70$

일반 도로 KITTI 장면에서는 정면에 큰 수직 벽이 항상 존재하지 않는다. 따라서 `front_wall`은 장면에 따라 적거나, 차량/가드레일/수풀 일부가 섞일 수 있다.

제 5 장

색상과 Rerun 출력

라벨별 색상은 `segment_color_rgb()`에 정의했다.

Label	Color	RGB	의미
floor	green	(70, 180, 90)	바닥 plane inlier
left_wall	blue	(60, 150, 255)	차량 좌측 벽 후보
right_wall	orange	(255, 160, 60)	차량 우측 벽 후보
front_wall	pink	(230, 80, 160)	차량 전방 벽 후보
other	gray	(150, 150, 150)	나머지 점

Rerun에서는 하나의 colored Points3D로 기록한다.

```
/lidar/segments
/metrics/lidar_floor_points
/metrics/lidar_left_wall_points
/metrics/lidar_right_wall_points
/metrics/lidar_front_wall_points
```

Rerun을 쓰지 않는 경우에는 같은 색상 정보를 가진 PLY를 저장한다.

```
<output>/lidar_segments/*.ply
```

제 6 장

코드 위치

파일/함수	역할
include/lidar/lidar_segmenter.hpp	point, label, option, result 타입 정의
src/lidar/lidar_segmenter.cpp	KITTI .bin 로드, RANSAC segmentation, PLY 저장, 색상 매핑
LidarSegmenter::process()	floor, left/right/front wall 라벨링 핵심 로직
segment_color_rgb()	Rerun/PLY에 들어가는 라벨별 색상
src/io/kitti_raw_reader.cpp	velodyne_points/data/*.bin path 수집
apps/run_vio.cpp	CLI 옵션, main loop 호출, Rerun logging
RerunLogger::log_lidar_segments()	/lidar/segments Points3D 기록

제 7 장

실행 명령

Rerun-enabled build:

```
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && cmake -S . -B build_verify_rerun -  
DCMAKE_BUILD_TYPE=Release -DENABLE_RERUN=ON && cmake --build build_verify_rerun -j$(nproc)'
```

전체 frame 실행:

```
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && mkdir -p results/  
kitti_raw_2011_09_26_drive_0117_full && ./build_verify_rerun/run_vio config/  
kitti_raw_2011_09_26_drive_0117_local_max_frame.yaml --segment-lidar --segment-every 1 --  
rerun-save results/kitti_raw_2011_09_26_drive_0117_full/lidar_segments_full.rrd --rerun-image  
-every 10'
```

Rerun 열기:

```
rerun D:\02_research\04_cpp_LC-EKF_VIO\results\kitti_raw_2011_09_26_drive_0117_full\  
lidar_segments_full.rrd
```

제 8 장

직접 수집한 데이터로 응용하기

현재 구현은 run_vio의 KITTI raw reader에 연결되어 있다. 따라서 가장 빠른 응용 방법은 직접 수집한 데이터를 KITTI raw와 비슷한 폴더 구조로 변환하는 것이다.

8.1 수집해야 하는 데이터

데이터	필요 이유
Stereo camera image	현재 VIO frontend가 left/right image를 필요로 한다.
IMU 또는 OXTS-like packet	현재 reader가 IMU 초기화와 propagation 입력을 필요로 한다.
LiDAR point cloud	floor/wall segmentation의 실제 입력이다.
Calibration YAML	camera intrinsics, stereo extrinsics, camera-IMU transform을 설정한다.

LiDAR segmentation 자체는 LiDAR 점만 있으면 가능하지만, 현재 실행 파일은 camera/IMU도 함께 로드한다. LiDAR만 단독으로 돌리고 싶다면 LidarSegmenter를 호출하는 별도 LiDAR-only executable을 추가하는 것이 더 깔끔하다.

8.2 권장 폴더 구조

직접 수집한 데이터를 다음처럼 정리한다. 파일명은 사전순 정렬이 곧 frame 순서가 되도록 10자리 zero padding을 권장한다.

```
data/my_dataset/my_drive_sync/  
  image_00/  
    timestamps.txt  
    data/  
      0000000000.png  
      0000000001.png  
  image_01/  
    timestamps.txt  
    data/  
      0000000000.png  
      0000000001.png  
  oxts/
```

```

timestamps.txt
data/
  0000000000.txt
  0000000001.txt
velodyne_points/
  data/
    0000000000.bin
    0000000001.bin

```

현재 KittiRawReader는 Velodyne timestamp 파일은 쓰지 않고, camera frame index와 같은 index의 .bin 파일을 사용한다. 즉, image_00/data/0000000042.png에 대응하는 LiDAR는 velodyne_points/data/0000000042.bin이라고 가정한다.

8.3 LiDAR bin 변환 규칙

각 LiDAR frame은 little-endian float32 네 개를 순서대로 저장한다.

```

float32 x
float32 y
float32 z
float32 intensity

```

좌표계는 KITTI Velodyne 기준으로 맞춘다.

축	직접 데이터 변환 규칙
x	차량/센서 전방이 양수
y	차량/센서 좌측이 양수
z	위쪽이 양수
단위	meter

예를 들어 사용하는 LiDAR SDK가 x right, y forward, z up 같은 좌표계를 낸다면, .bin 저장 전에 KITTI 좌표계로 회전/축 교환을 해야 한다.

8.4 Timestamp와 동기화

현재 reader는 timestamp line에서 시각 부분을 읽는다.

```
2011-09-26 15:38:03.123456789
```

직접 데이터에서는 절대 날짜보다 frame 순서와 sensor 간 동기화가 중요하다.

- camera left/right frame 수가 같아야 한다.
- Velodyne .bin frame 수가 camera frame 수와 같거나 더 많아야 한다.
- 현재 구현은 nearest timestamp matching이 아니라 같은 index matching을 사용한다.
- frame drop이 있으면 image와 LiDAR 파일명을 같은 index로 다시 정렬한다.

8.5 Config 작성

기존 config를 복사해서 dataset/output/calibration을 바꾼다.

```
Copy-Item `
  D:\02_research\04_cpp_LC-EKF_VIO\config\kitti_raw_2011_09_26_drive_0117_local_max_frame.yaml `
  D:\02_research\04_cpp_LC-EKF_VIO\config\my_drive.yaml
```

수정할 항목:

```
dataset_type: "kitti_raw"
dataset: "/mnt/d/02_research/04_cpp_LC-EKF_VIO/data/my_dataset/my_drive_sync"
output: "/mnt/d/02_research/04_cpp_LC-EKF_VIO/results/my_drive"
max_frames: 0
```

그리고 cam0, cam1의 intrinsics와 T_cam_imu를 직접 장비 calibration으로 바꾼다.

8.6 실행

```
wsl bash -lc 'cd /mnt/d/02_research/04_cpp_LC-EKF_VIO && mkdir -p results/my_drive && ./
  build_verify_rerun/run_vio config/my_drive.yaml --segment-lidar --segment-every 1 --rerun-
  save results/my_drive/lidar_segments.rrd --rerun-image-every 10'
```

Rerun으로 확인한다.

```
rerun D:\02_research\04_cpp_LC-EKF_VIO\results\my_drive\lidar_segments.rrd
```

8.7 현장 수집 체크리스트

- 차량/센서가 멈춰 있을 때 몇 초간 기록해서 IMU 초기화가 안정되게 한다.
- LiDAR가 바닥을 충분히 보도록 설치한다. 너무 하늘/전방만 보면 floor RANSAC이 약해진다.
- 좌우 벽을 보고 싶다면 도로 가장자리, 복도, 주차장 벽처럼 수직 구조가 있는 장면을 포함한다.
- frame drop이 생기면 camera/LiDAR index alignment를 먼저 고친다.
- Rerun에서 '/lidar/segments'가 기울어져 보이면 LiDAR 좌표계 변환이 잘못된 것이다.

제 9 장

현재 한계

- geometry 기반이므로 semantic class를 이해하지 않는다.
- 벽과 비슷한 수직 구조물(차량 측면, 가드레일, 수풀)이 wall 후보로 섞일 수 있다.
- 각 frame을 독립적으로 처리하므로 temporal smoothing은 없다.
- 현재 segmentation 결과는 EKF update에 사용하지 않고 시각화/분석용으로만 사용한다.
- 현재 실행 파일은 KITTI-like stereo/IMU/LiDAR 구조를 가정한다. 임의 LiDAR-only 로그를 바로 읽지는 않는다.